





Hands-on: RESTful APIs with Python

A DevCouRSE Workshop

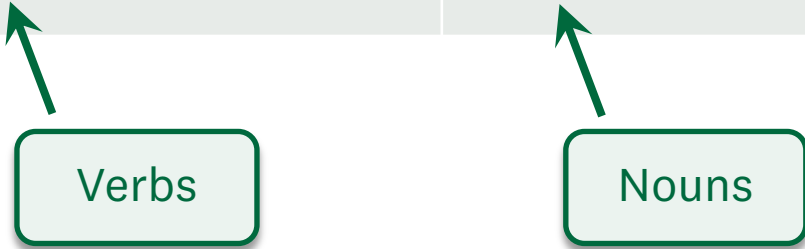
Tim Meehan, Simon Stone

Research Computing @ ITC, Dartmouth College



REST Actions <-> HTTP Methods

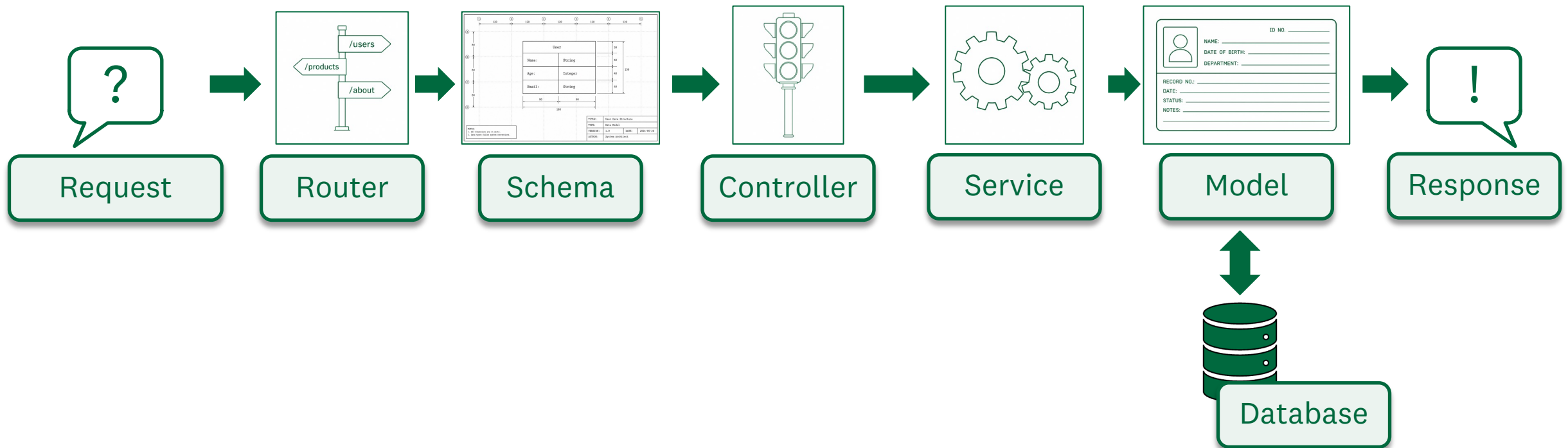
REST Action	HTTP Method	Example Endpoint	Purpose
Create	POST	/users	Create a new resource
Read	GET	/users/123	Retrieve specific resource
Update	PUT	/users/123	Update specific resource
Delete	DELETE	/users/123	Remove specific resource



Verbs

Nouns

Life-cycle of a typical backend API request



Today's Project: Dartmouth Places

-  An API to manage a directory of places of interest at Dartmouth
-  List all Places
-  List a specific Place
- Create a Place
-  Update a Place
-  Delete a Place
-  Only allow authenticated access

Place

Name
Location
Description
Category
Noise Level
Outlets?
WiFi?
Capacity
Public?

Implementing RESTful APIs in Python

Today's stack:

- **FastAPI:** Handling routers, controllers, HTTP request mapping
- **SQLModel:** Handling schemas, models, and database I/O
- **Plain Python:** Implementation of services
- **SQLite:** database engine





Let's get into it!



Setup

- 🐙 Clone repo (if you fork, fork all branches!):
<https://github.com/dartmouth/devcourse-python-api-handson>
- 🚀 In the project root: `uv sync`
- 🐾 A reference implementation of each step is in a separate branch



Step 0: Boilerplate





Step 1: Health check





Step 2: Places





Step 2: Places

! No database yet!

- In `schemas.py`:
 - Define a `Place` schema
- In `routers/places.py`:
 - Create a list with a few mocked-up `Place` objects
 - Create a route to return the whole list
 - Create a route to return a `Place` in that list with a specific ID



Step 3: Database





Step 3: Database

- In `models.py`:
 - Turn `Place` schema into a model
- In `schemas.py`:
 - Add `PlaceResponse` and `PlaceCreate` schemas
- In `database.py`:
 - Add a function to create the database and tables
 - Add a session factory function
- In `main.py`:
 - Call database creation function as part of the API app's lifespan
- In `routers/places.py`:
 - Refactor `/places` and `/places/{place_id}` to use database
 - Add route to create a `Place`
 - Add route to update a `Place`
 - Add route to delete a `Place`



Step 4: Auth





Step 4: Auth

- In `middleware/auth.py`:
 - Implement a function to verify a bearer token
- Use dependency injection to protect endpoints:
 - Protect the entire `/places` router
 - **Bonus:** Protect only the routes that write to the database



Thank you